

Jogo da Vida de Conway

Braian Melo e Leonardo Ribeiro

23 de novembro de 2025

1 Introdução

Este trabalho tem como objetivo implementar o do Jogo da Vida de Conway de duas maneiras: uma sequencial e outra paralela. Ao longo deste documento, iremos explicar como foi realizada a implementação de cada método, as decisões técnicas tomadas e os resultados obtidos na comparação entre os dois métodos.

2 O Jogo da Vida de Conway

O Jogo da Vida é um autômato celular, ou seja, um sistema evolutivo definido por regras simples que determinam o estado futuro de cada célula. Ele foi criado em 1970 por John H. Conway, de onde deriva o nome “Jogo da Vida de Conway”, um matemático britânico bastante influente do século passado.

Esse jogo tem como objetivo reproduzir alterações em um conjunto de seres vivos. Sua definição é bastante simples: considera-se uma matriz bidimensional em que cada célula pode estar viva ou morta, e sua evolução ocorre conforme regras que determinam o nascimento, a sobrevivência ou a morte das células a cada geração.

Ao longo de cada geração, cada célula é analisada com base em seus vizinhos, e as regras abaixo são aplicadas para definir o estado da próxima geração.

- **Regra 1 — Subpopulação:** uma célula viva morre se possuir **menos de 2** vizinhos vivos.
- **Regra 2 — Superpopulação:** uma célula viva morre se possuir **mais de 3** vizinhos vivos.
- **Regra 3 — Sobrevivência:** uma célula viva permanece viva se possuir **2 ou 3** vizinhos vivos.
- **Regra 4 — Nascimento:** uma célula morta torna-se viva se possuir **exatamente 3** vizinhos vivos.

Abaixo, nas figuras 1 e 2, temos um exemplo visual de como o jogo funciona.

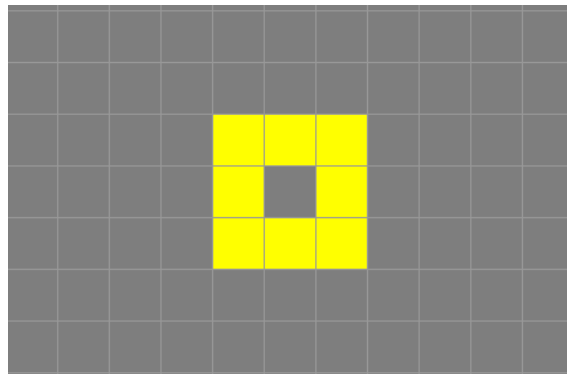


Figura 1: Na 1ª geração temos uma forma inicial parecida com a de um quadrado.

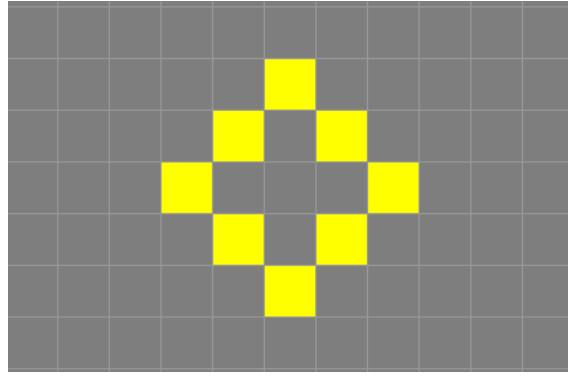


Figura 2: Na 2ª geração, enquanto as células dos cantos sobrevivem, as das laterais morrem por superpopulação; células adjacentes nascem.

3 Implementação

3.1 Requisitos e Entrada do Usuário

Para a execução do programa, são necessários alguns requisitos básicos. O sistema deve possuir o compilador `gcc` instalado, com suporte à biblioteca `pthread`. Além disso, é recomendável que a máquina disponha de múltiplos núcleos de processamento para que o paralelismo obtido pelas threads seja efetivo.

O programa recebe como parâmetro de entrada um caminho de arquivo *PATH*, que indica o arquivo a ser lido. Além disso, o usuário deve especificar o modo de execução desejado: `seq` para o modo sequencial e `par` para o modo paralelo.

Um exemplo de execução no terminal é apresentado a seguir:

```
$ ./Programa entrada.txt par
```

Neste exemplo, o programa é executado a partir da pasta raiz e utiliza o arquivo *entrada.txt* como entrada, sendo executado no modo paralelo. Ao final da simulação, os resultados são gravados em um arquivo de saída chamado *saida.txt*.

3.1.1 Arquivo de entrada

Para que o programa seja executado corretamente, o arquivo de entrada deve estar formatado da seguinte maneira:

- **N GEN**: número de gerações da simulação.
- **L e C**: número de linhas e colunas da matriz.
- **N**: quantidade de células vivas iniciais.
- Em seguida, **N linhas**, cada uma contendo as coordenadas **X** e **Y** de uma célula viva.

Abaixo, temos um exemplo de arquivo de entrada:

```
50
10 10
5
0 1
2 3
4 4
5 5
8 9
```

Ao final da execução, o arquivo *saida.txt* será gerado seguindo a mesma formatação do arquivo de entrada.

3.2 Método sequencial

Para a implementação do algoritmo, foi necessário definir uma estrutura de dados capaz de armazenar a matriz de células. A solução mais direta seria alocar uma matriz $M \times N$ para cada geração, copiando os estados e desalocando a matriz anterior. No entanto, essa abordagem geraria um *overhead* significativo devido às constantes operações de alocação e liberação de memória.

Para evitar esse custo, adotamos uma estrutura em que cada célula possui duas variáveis internas: uma representando o estado atual e outra o estado da próxima geração. Dessa forma, a cada iteração utilizamos uma das camadas para leitura e a outra para escrita, invertendo os índices ao final de cada geração. Esse método elimina realocações desnecessárias e torna a simulação mais eficiente.

Abaixo está o código referente a essa estrutura:

```
typedef struct Celula {
    bool estah_vivo[2];
} Celula;

typedef struct Tabuleiro {
    int linhas;
    int colunas;
    Celula **celulas;
    bool indice;
} Tabuleiro;
```

A parte referente à estrutura do jogo ficou em outro arquivo, o *Jogo-Vida-Conway.c*. Esse módulo é responsável por armazenar as gerações e executar o algoritmo sequencial. Nesse processo, o programa percorre o tabuleiro célula por célula, verifica a quantidade de vizinhos vivos e aplica as regras do Jogo da Vida para definir o estado de cada célula na geração seguinte.

3.3 Método paralelo

A partir da análise do método sequencial, observou-se que seria possível dividir a matriz entre várias *threads*, de forma que cada uma ficasse responsável pelo cálculo de uma região específica do tabuleiro. Essa divisão permite explorar o paralelismo inerente ao problema, acelerando o processamento de cada geração ao distribuir o trabalho entre múltiplas unidades de execução.

O código paralelo segue exatamente a mesma lógica do sequencial, garantindo uma comparação justa entre os dois métodos. Entretanto, foi necessário introduzir uma estrutura adicional para organizar as informações compartilhadas entre as *threads*. Essa estrutura auxilia na distribuição das tarefas, repassando a cada *thread* os limites das linhas sob sua responsabilidade durante o cálculo da geração atual.

```
typedef struct {
    JogoParalelo *jogo;
    int linha_inicio;
    int linha_fim;
} ThreadArgs;
```

3.4 Dificuldades encontradas

Ao realizar o método paralelo, identificamos um desafio: as células nas fronteiras dependem do estado de vizinhas que podem pertencer a outra *thread*, o que exige comunicação e sincronização entre elas. Para evitar que alguma *thread* avance demais em relação às outras e garantir que todas estejam atuando sobre a mesma geração, optamos por criar uma barreira que faça com que cada *thread* avance igualmente na avaliação.

Outro ponto importante que vale ser destacado é que, durante o desenvolvimento, identificamos problemas significativos de *cache misses* e *cache invalidation*. Por isso, optamos por dividir a matriz

por linhas, e não por blocos ou sessões. Essa escolha permitiu que cada *thread* acessasse regiões contíguas da memória, melhorando a localidade espacial e reduzindo a frequência de *cache miss*.

4 Resultados

4.1 Hipóteses iniciais

Antes de realizar os testes com o *Gprof*, esperávamos que o algoritmo paralelo apresentasse melhor desempenho em tabuleiros maiores, pois o método sequencial possui complexidade $\mathbf{O}(M \times N)$, onde M é o número de linhas e N o número de colunas do tabuleiro.

4.2 Resultados encontrados

Seguindo nosso pressuposto, iniciamos com um teste em um tabuleiro de 100x100 com 200 células inicialmente vivas. Os resultados estão apresentados na figura 3.

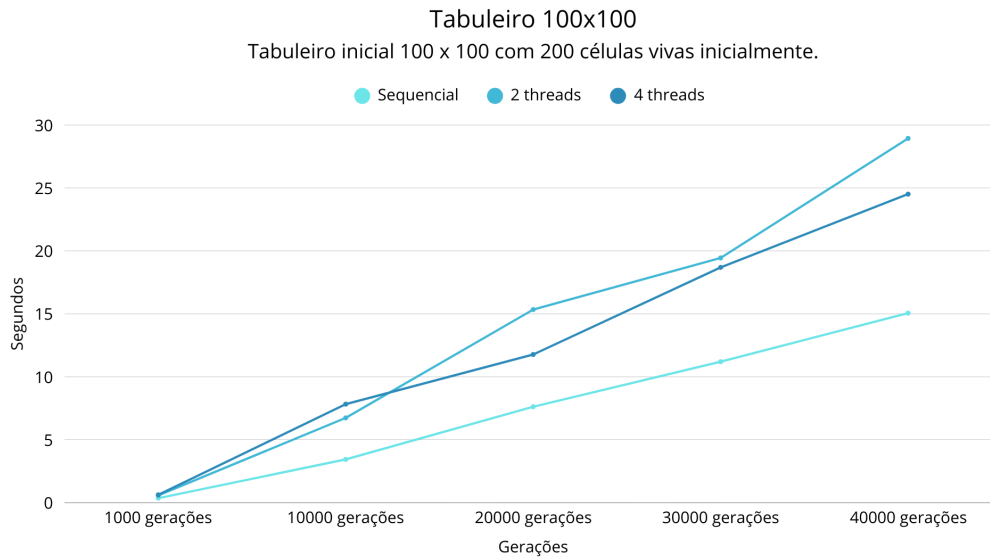


Figura 3: Primeiro teste realizado.

Como é possível observar, o tempo do método sequencial se manteve praticamente linear e ficou significativamente abaixo do tempo dos métodos paralelos.

No segundo teste, para avaliar nossa hipótese de que o aumento do tamanho do tabuleiro favoreceria o método paralelo, utilizamos um tabuleiro de 1000x1000 com 5000 células vivas inicialmente. Os resultados estão ilustrados na figura 4.

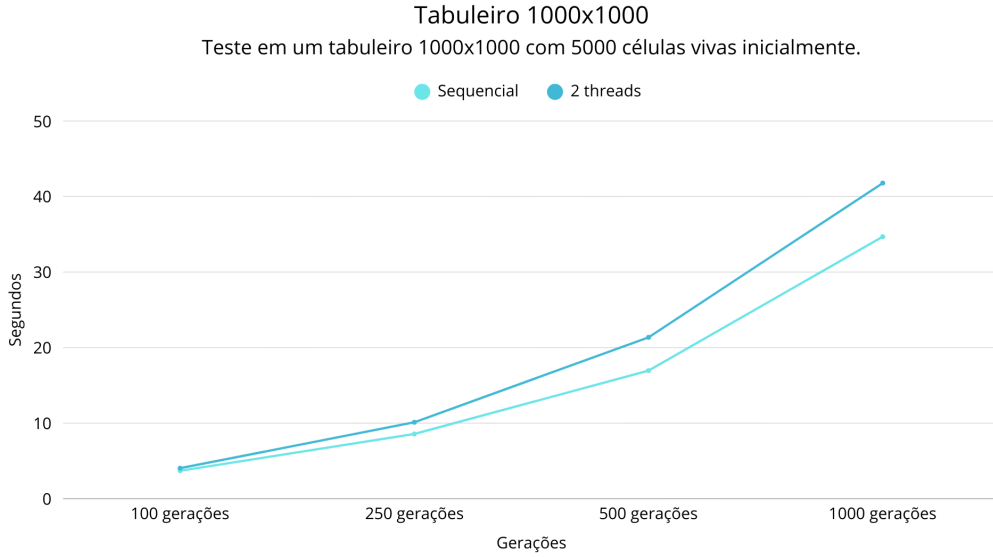


Figura 4: Resultados do segundo teste realizado.

Com os resultados inesperados, optamos por realizar um último teste. Neste caso, mantivemos o número de células vivas iniciais $N_{\text{vivas}} = 500$ e o número de gerações $G = 500$, enquanto a variável analisada foi o tamanho do tabuleiro $M \times N$. Na figura 5 tem os resultados deste teste.

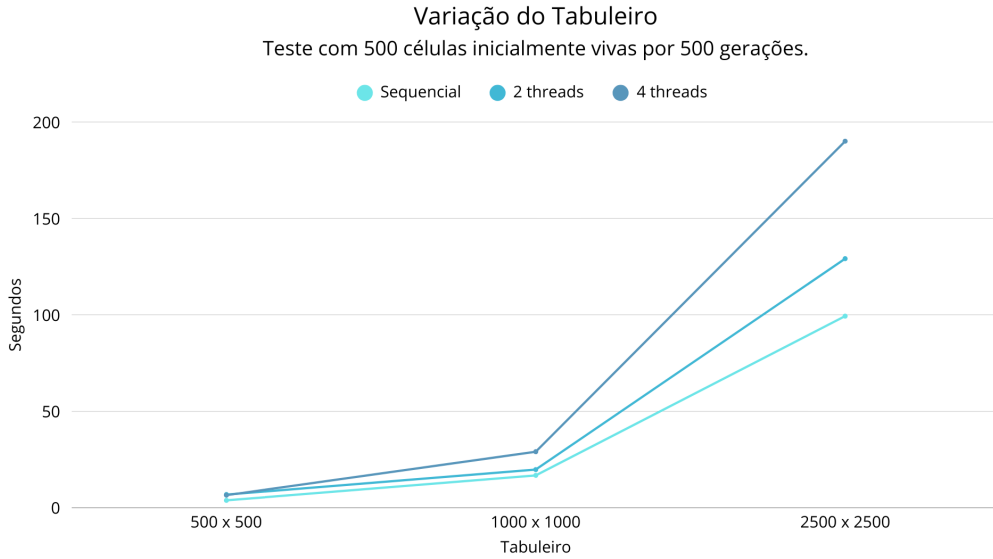


Figura 5: Resultado de teste com o tamanho do tabuleiro variando

4.3 Análise do desempenho

Com os resultados encontrados, percebemos que o método sequencial apresentou tempos de execução menores do que os métodos paralelos, mesmo em *grids* maiores. Esse comportamento pode ser explicado por alguns fatores:

- **Criação e gerenciamento de *threads*:** Cada *thread* adiciona custos de criação e sincronização, principalmente devido às barreiras que garantem que todas as *threads* avancem simultaneamente a cada geração.
- **Invalidação da linha de *cache* e *cache miss*:** Na execução paralela, múltiplas *threads* acessam e atualizam linhas adjacentes do tabuleiro simultaneamente. Isso provoca invalidação das linhas de cache entre os núcleos da *CPU* e aumenta o número de cache misses em relação ao método sequencial, prejudicando o desempenho.

- **Tamanho do problema relativamente pequeno:** Para tabuleiros menores, o custo fixo da paralelização supera os ganhos de divisão de trabalho, tornando o método sequencial mais rápido.

Ao pesquisar o assunto em outras fontes, identificamos que existem abordagens capazes de tornar o código paralelo mais rápido que o sequencial. Uma delas consiste em criar uma lista contendo as células inicialmente vivas e seus vizinhos. Após mapear essas células, o algoritmo divide o vetor igualmente entre as threads. Essa estratégia reduz o número de operações desnecessárias e diminui o número de cache misses e invalidações de linhas de cache.

Outro método interessante consiste em iniciar o processamento de forma sequencial, mas, assim que a primeira *thread* termina de calcular uma parte da matriz, outra *thread* inicia o cálculo da geração seguinte.

Entretanto, como nossa intenção era aplicar a técnica de **decomposição de domínio** na matriz, optamos por não implementar esses outros métodos. Além disso, como a paralelização não apresentou ganhos, decidimos não gerar os gráficos de *speedup* e *eficiência*.

5 Conclusão

Neste trabalho, implementamos e analisamos o Jogo da Vida de Conway utilizando abordagens sequencial e paralela. Inicialmente, esperávamos que a paralelização proporcionasse ganhos de desempenho em tabuleiros maiores, devido à divisão do trabalho entre múltiplas *threads*. No entanto, os resultados mostraram que o método sequencial apresentou tempos de execução menores em todos os testes. Fatores como o custo de criação e sincronização de *threads*, o aumento de *cache misses* e a *cache invalidation* no método paralelo explicam esse comportamento.

Dessa forma, o trabalho destacou os desafios da paralelização em problemas com dependência de dados e evidenciou que nem sempre a paralelização compensa. Em certos casos, estratégias diretamente adaptadas do modelo sequencial podem não ser eficientes quando aplicadas em paralelo, sendo necessário repensar a organização do algoritmo e a forma de dividir o trabalho para realmente obter ganhos de desempenho.

Referências

- KOZERA, Stefanie. **Parallel Conway's Game of Life**. Disponível em: https://www.secs.oakland.edu/~llamocca/Courses/ECE4772/F24/FinalProject/Group2_Conway_presentation.pdf. Acesso em: 22 nov. 2025.
- FOWLER, Adam. **ParLife: A Parallel Implementation of Conway's Game of Life**. Disponível em: <https://www.cs.columbia.edu/~sedwards/classes/2021/4995-fall/reports/ParLife.pdf>. Acesso em: 22 nov. 2025.